UNIVERSIDAD PERUANA
CAYETANO HEREDIA

Entregable N°11

E11: Metodología VDI - Integración

Caso clínico 4 - Secuela de parálisis braquial obstétrica

Munguia Avila, Ariana Alessandra
Martel Vilca, Dominick Pavel
Monzón La Rosa, María Isabel
Espinoza Cuya, Luis Diego
Guardamino Velasquez, Giovanni Jesus
Huanca Benites, Rodrigo Alonso

Lima – Perú 2026

1. **Integración física del prototipo**

● Presentar fotografías del prototipo ensamblado

A. Subsistemas Estructural:

- Placa Dorsal: Elemento central de soporte modelado con refuerzos triangulares y pestañas de fijación, diseñado para optimizar la distribución de las cargas mecánicas y de torsión sobre el tórax. Incorpora alojamientos adaptados para el acoplamiento de los servomotores de alto torque del subsistema de actuación. Posee un sistema de sujeción por correas conectando con la parte frontal.

- Placa Frontal: Componente diseñado para equilibrar el centro de gravedad del sistema ortopédico, nivelando las cargas distribuidas entre el plano dorsal y el plano frontal. Actúa como interfaz mecánica de conexión con el miembro superior mediante un sistema de sujeción por correas que se acoplan a través de las aberturas presentes en la placa.

- Ranuras de Anclaje y Sujeción: Aberturas perimetrales de baja fricción optimizadas para el guiado, ajuste y distribución uniforme de la tensión de las correas de sujeción ergonómica, asegurando un acoplamiento seguro y anatómicamente estable al cuerpo del usuario.

- Soporte de antebrazo: Componente estructural de geometría ergonómica y superficie cóncava, desarrollado mediante secciones elípticas en modelado CAD 3D para adaptarse al contorno del antebrazo. Incorpora aberturas laterales e inferiores para la conexión de correas de sujeción. Su función es proporcionar apoyo y estabilidad al miembro superior durante la rehabilitación, permitiendo su acoplamiento al mecanismo de actuación para asistir los movimientos de flexión y extensión del codo.

- Caja de circuitos: Conjunto estructural compuesto por dos piezas complementarias, desarrolladas mediante modelado CAD 3D. La primera corresponde a una carcasa rectangular con esquinas redondeadas, diseñada para alojar los circuitos electrónicos, la batería y el módulo de carga; mientras que la segunda incorpora las aberturas destinadas a los conectores externos, incluido el puerto USB. Su función es proteger y organizar los componentes electrónicos del prototipo, facilitando el acceso a las conexiones para la carga y comunicación del sistema.

- Carrete: Componente tipo polea, modelado con dos discos laterales y una garganta central para guiar el cable. En el esquema de integración se consideran dos carretes: uno pequeño asociado a la flexión del hombro y otro para conducir el cable a lo largo del miembro superior. Su función es mantener el recorrido del cable y transmitir la tracción del mecanismo de asistencia; la disposición mostrada es referencial y podrá ajustarse durante el montaje.

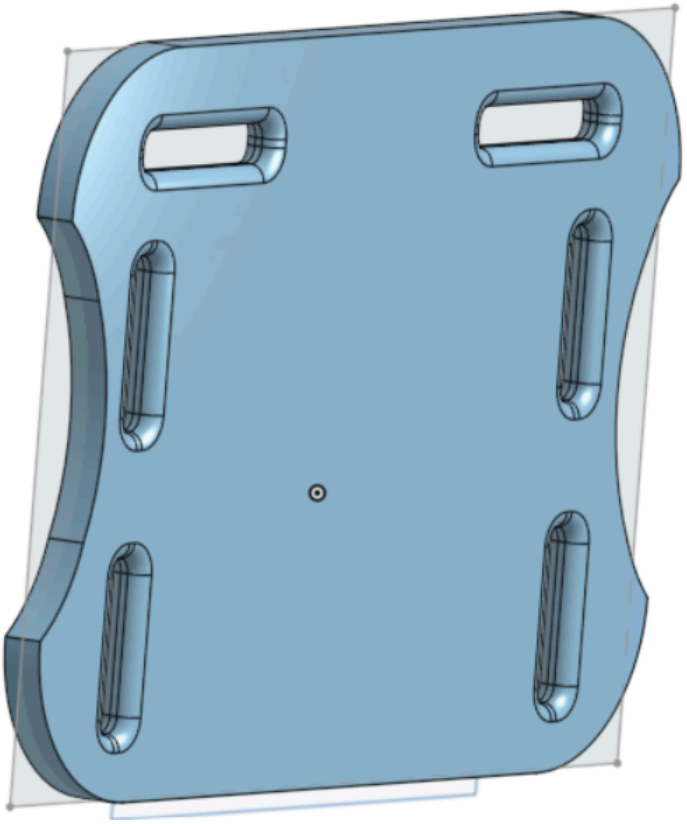
B. Subsistema de Actuación y Potencia:

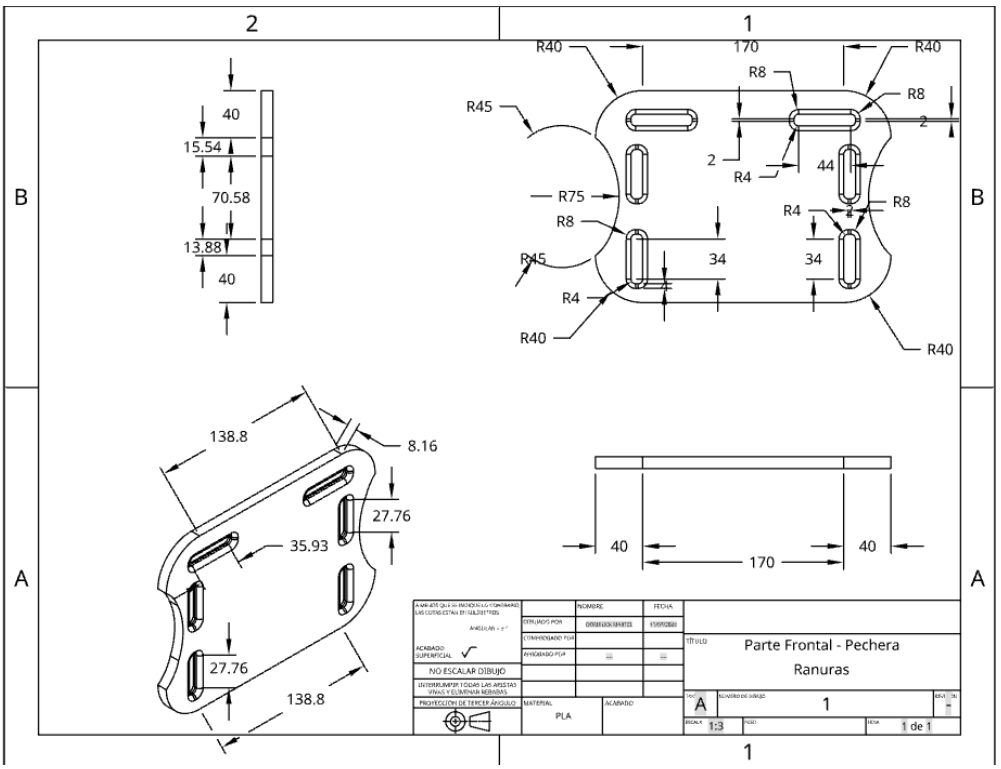
- Soporte de Servomotores: Alojamientos de geometría rectangular integrados rígidamente a la placa dorsal, diseñados para poder albergar los servomotores de alto torque encargados de la tracción y liberación controlada del cable de asistencia.

- Pestañas de fijación: Orejas de acoplamiento lateral dotadas de agujeros pasantes y refuerzos poligonales, optimizadas para garantizar una sujeción mecánica firme, estable y con alta resistencia.

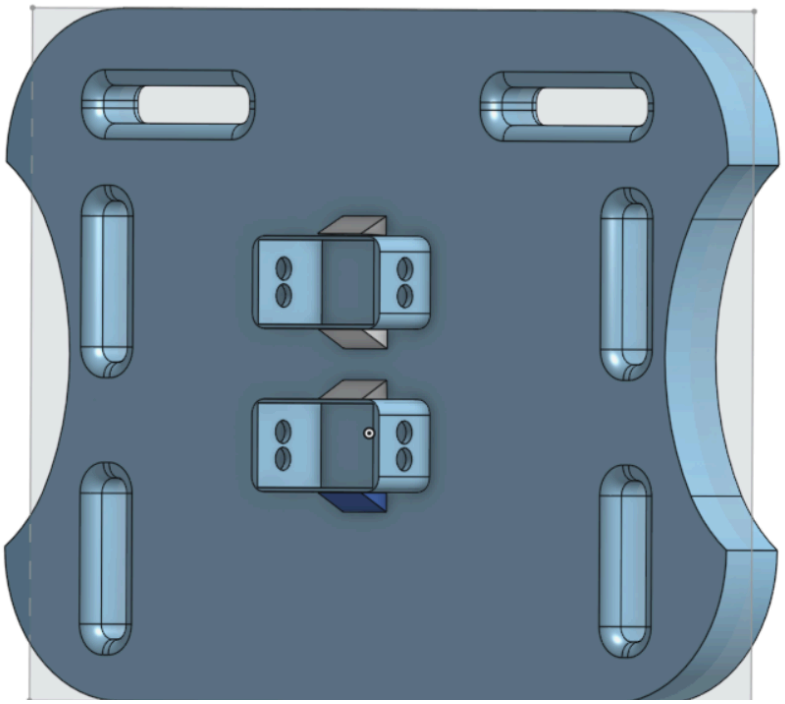
Planos:

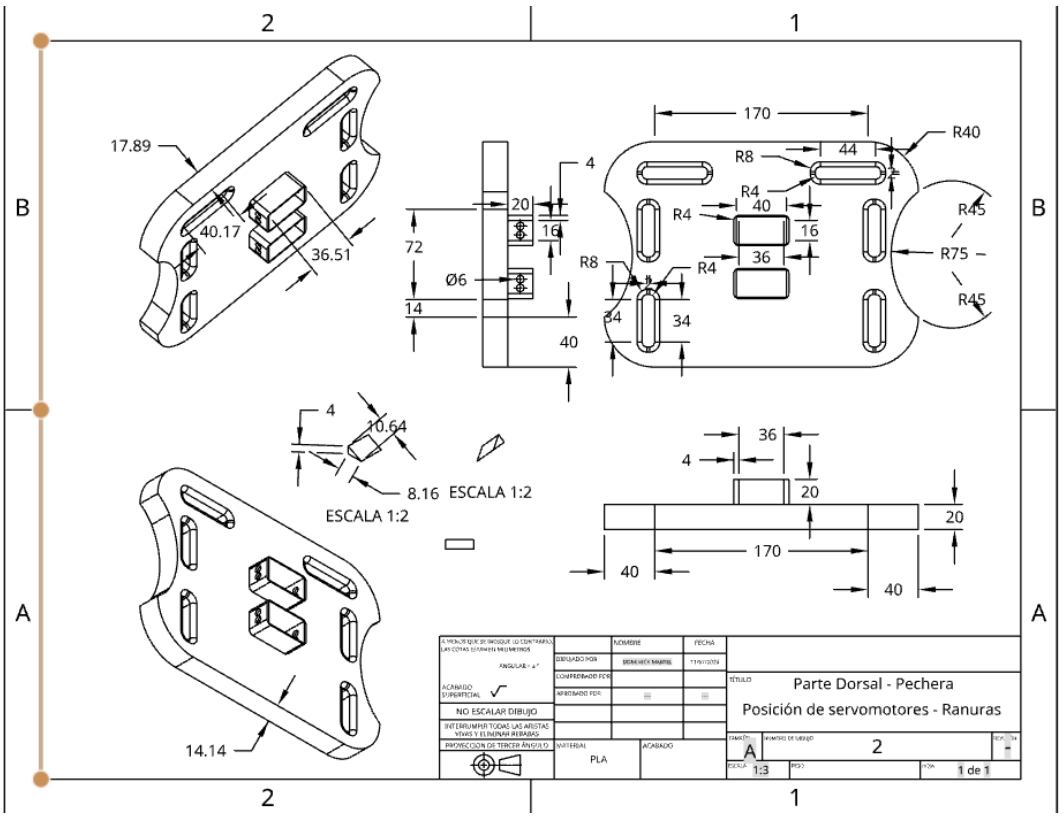
- Pechera - Parte Frontal



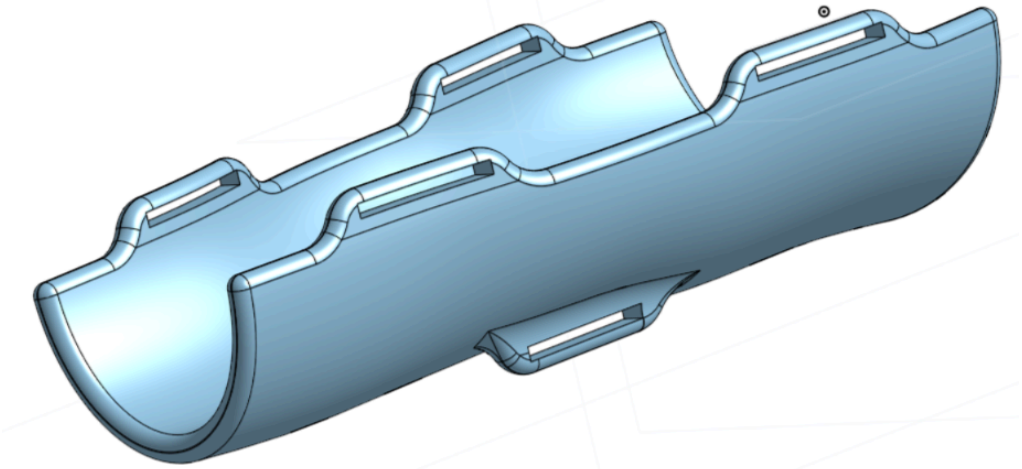


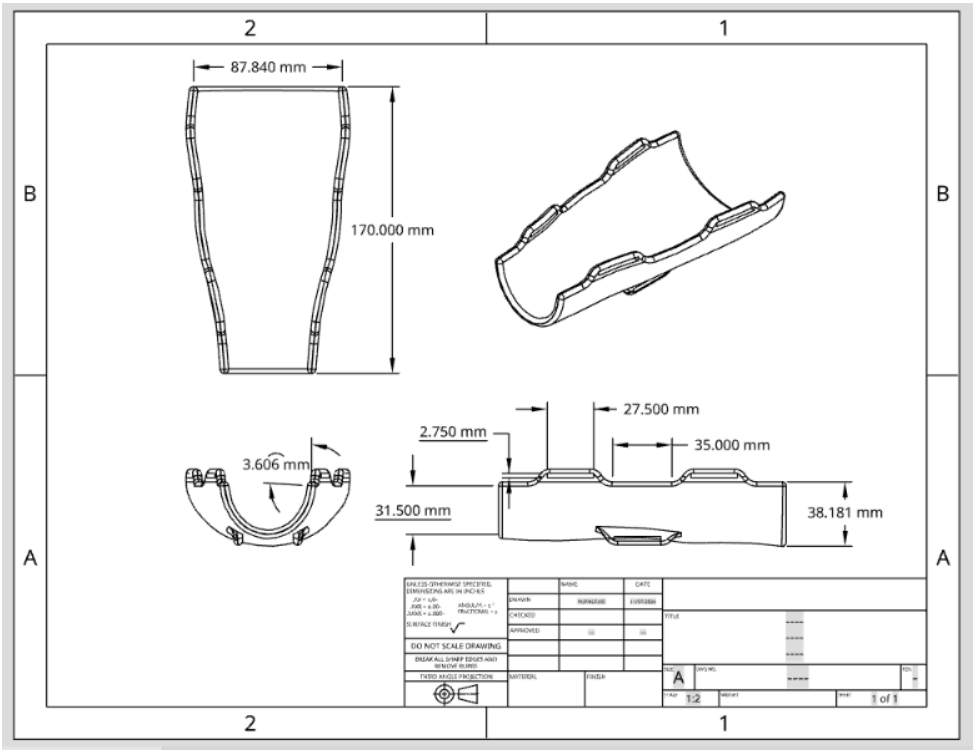
- Pechera - Parte Dorsal



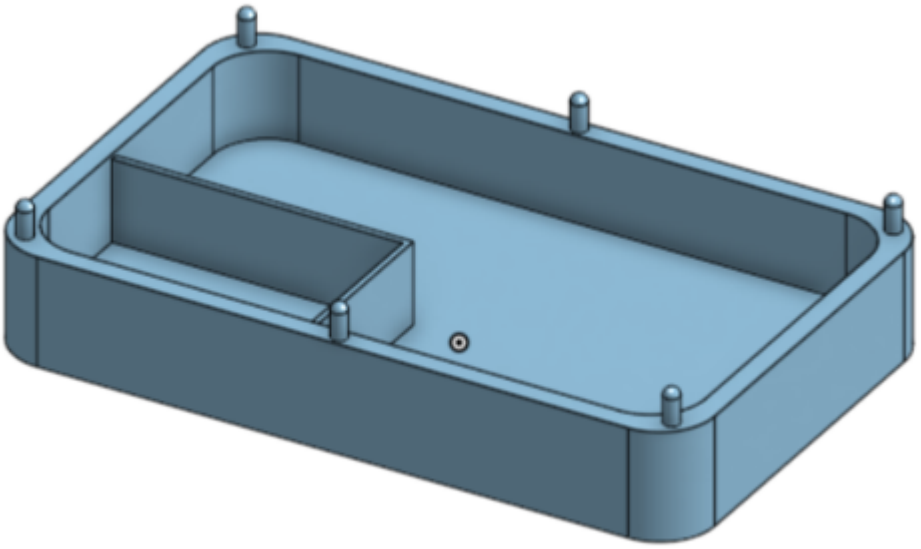


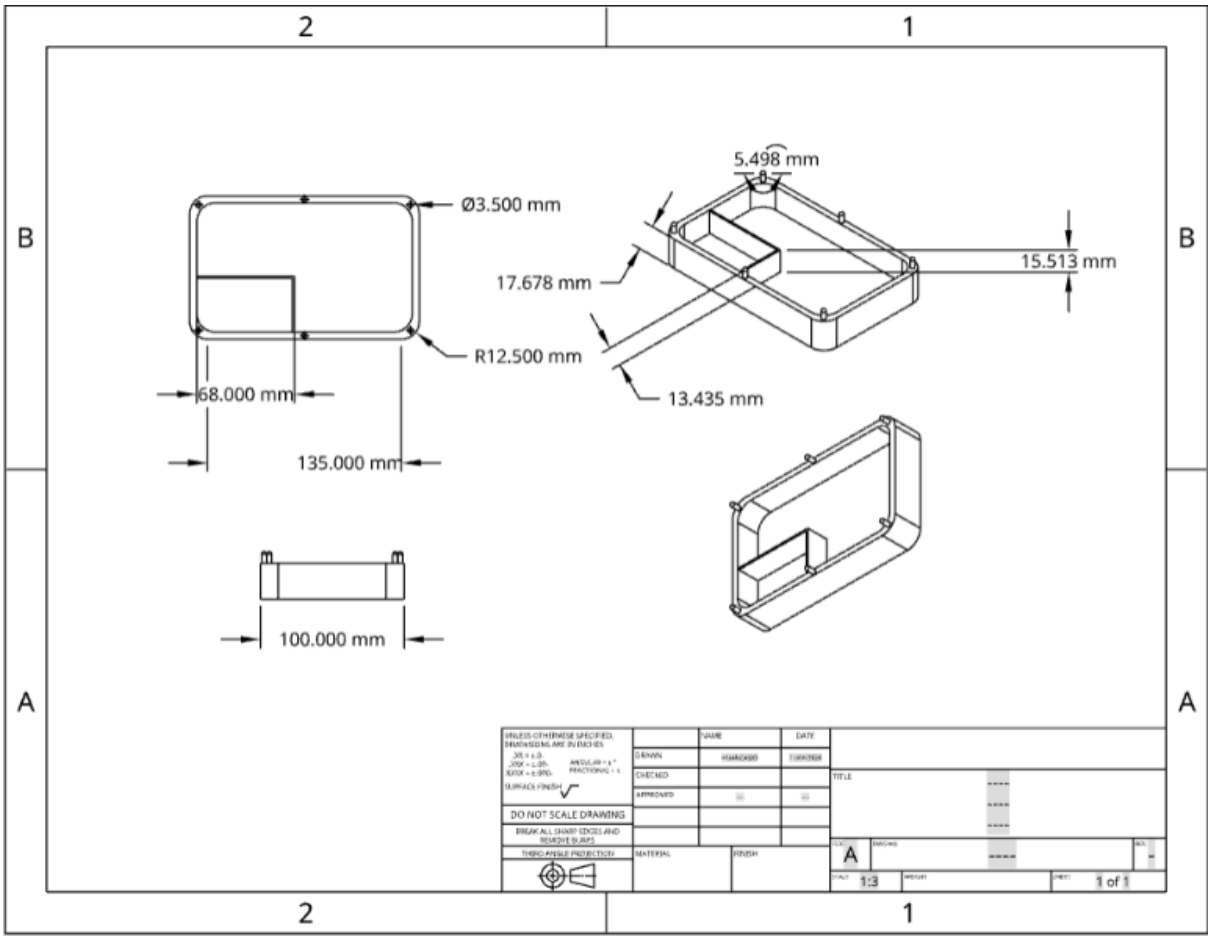
- Soporte de antebrazo:



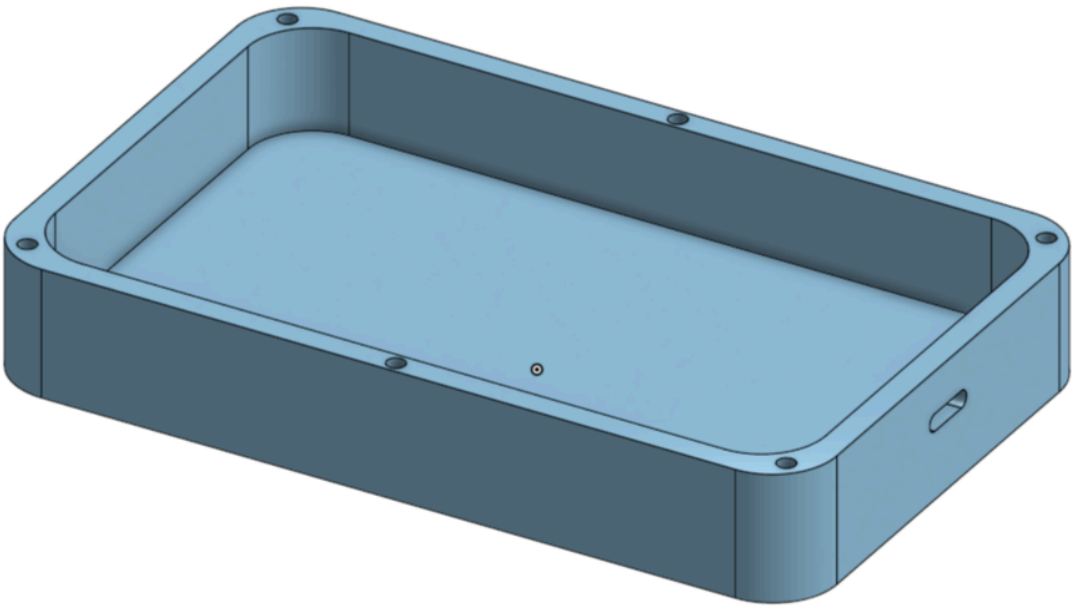


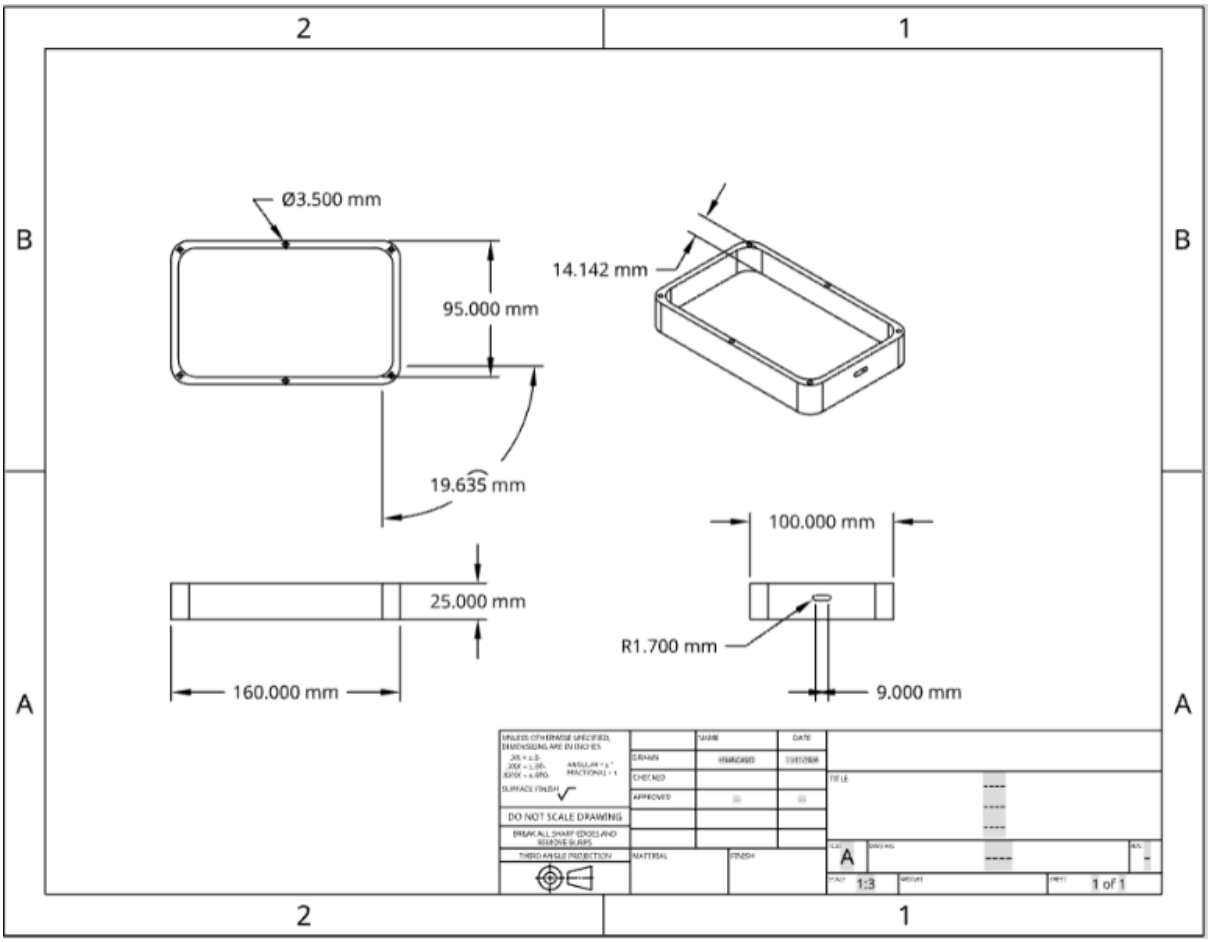
- Caja de circuitos:
  parte superior



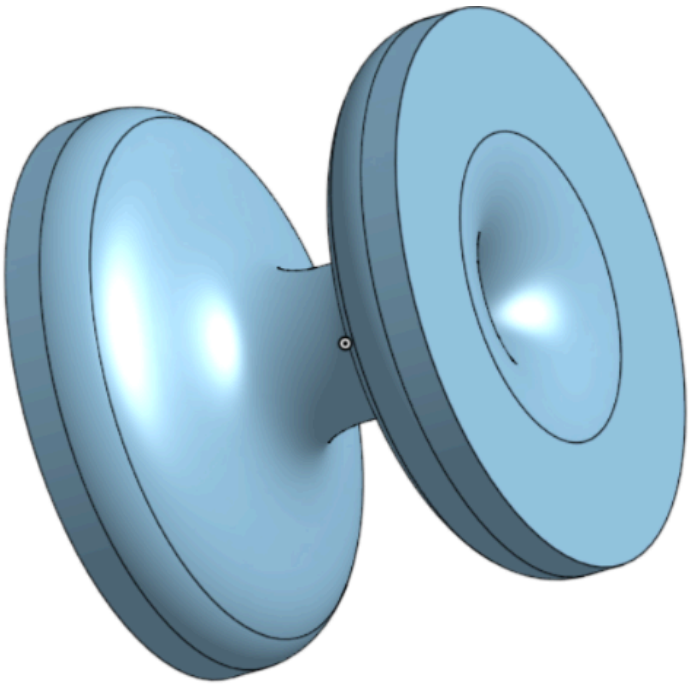


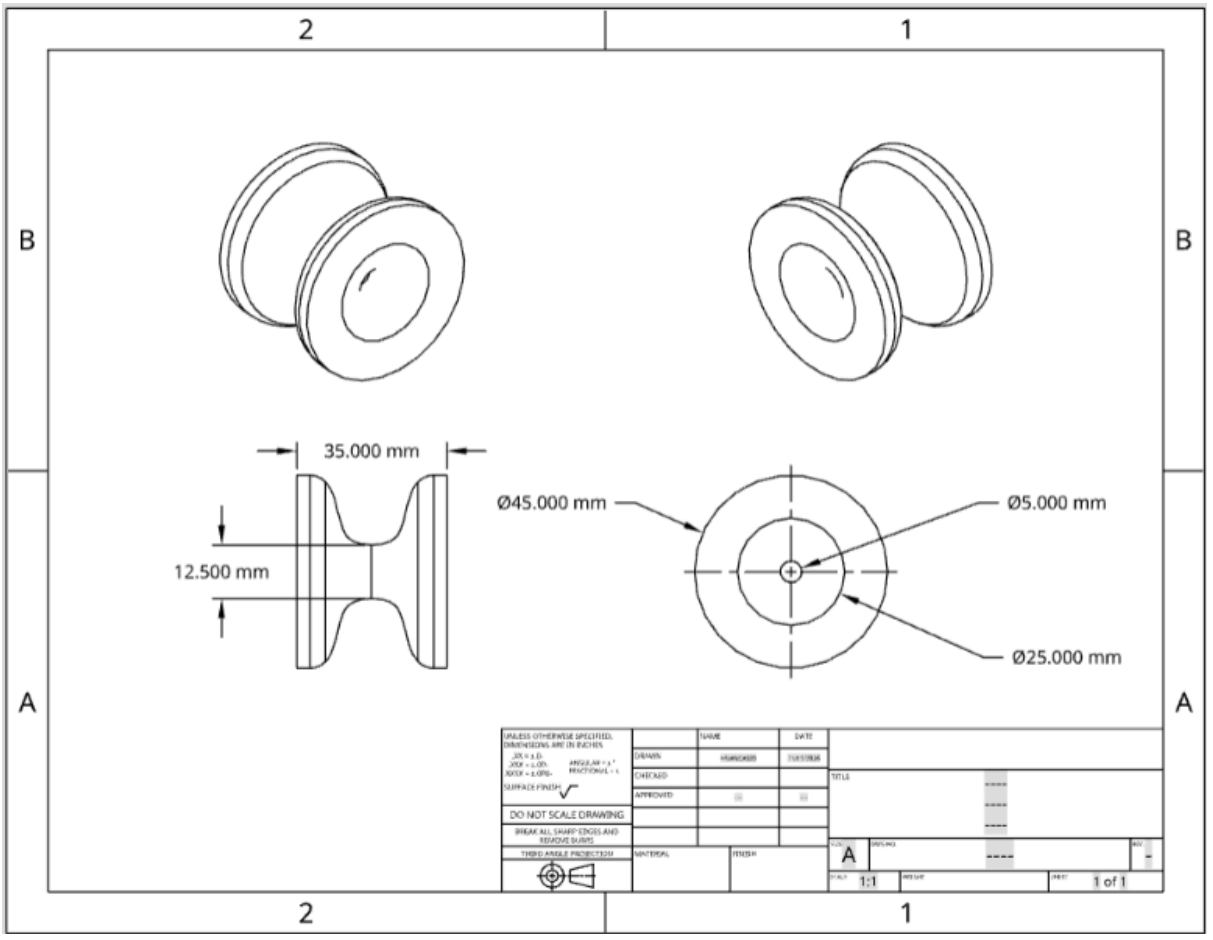
Parte inferior




2
1
B
A
Ø3.500 mm
95.000 mm
19.635 mm
14.142 mm
25.000 mm
160.000 mm
100.000 mm
R1.700 mm
9.000 mm
DO NOT SCALE DRAWING
1:3
1 of 1

- Carrete:





2. Funcionamiento del sistema

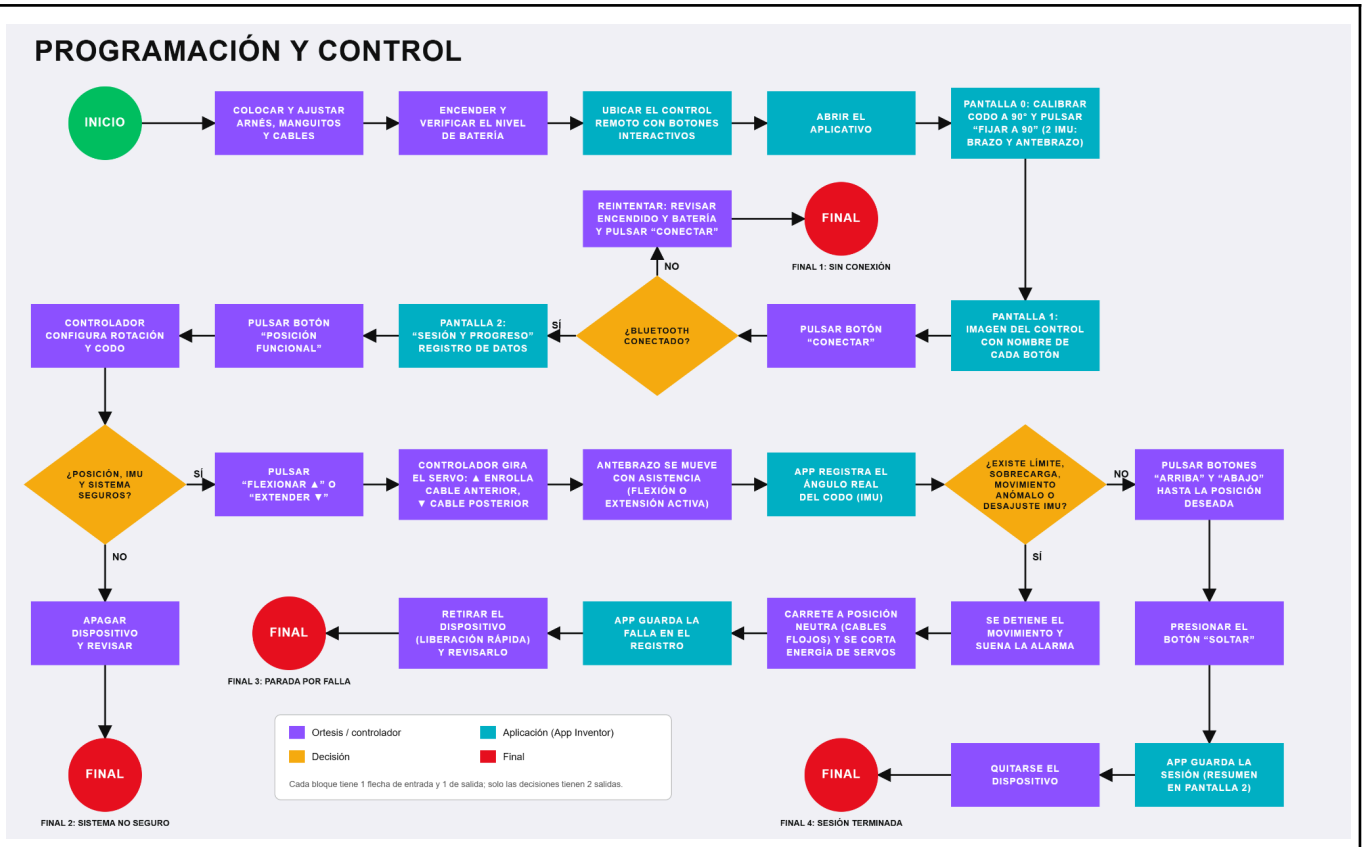


Diagrama 1: Lógica de funcionamiento

A. DIAGRAMA ESQUEMÁTICO

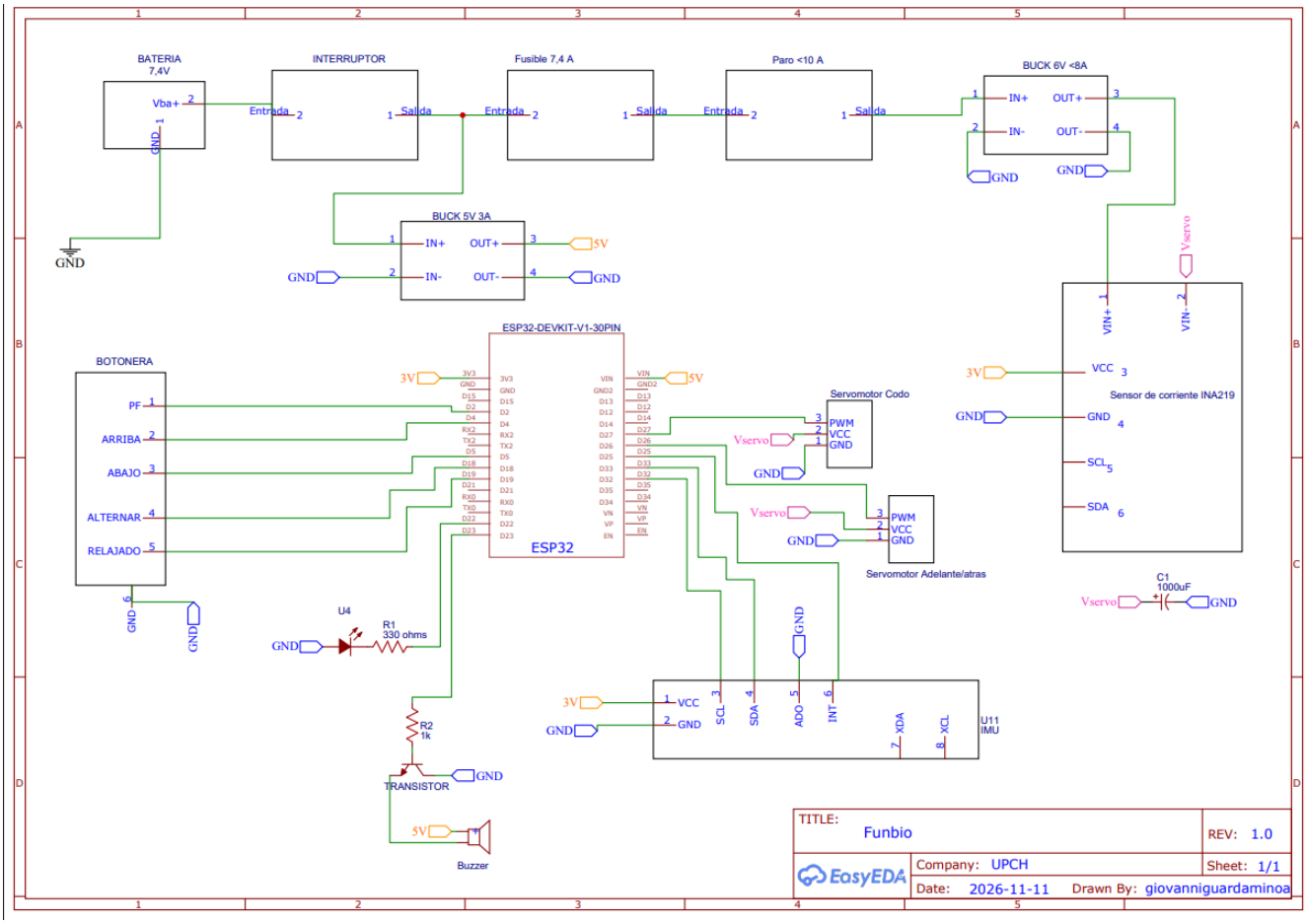


Diagrama 2: Lógica del esquemático

B. CÓDIGO DE PROGRAMACIÓN

Función	Captura
Cabecera y pines Define qué librerías se usan, que el programa solo compila para un ESP32 con Bluetooth Classic, a qué pin va cada elemento, la lista de estados y los objetos de hardware. **Qué hace el código:** Comentarios de cabecera: resumen de la botonera, de los comandos Bluetooth y de las librerías necesarias. Includes: Wire (I2C), BluetoothSerial, Preferences (memoria NVS), Adafruit INA219 y MPU6050, ESP32Servo. Comprobación en compilación (#error): si la	

```
// ---------- Variables de control ----------
int limiteHombroMax = 80;    // valores por defecto, se redefinen en calibración
int limiteCodoMax   = 45;
int anguloHombro = 0;
int anguloCodo   = 0;
bool controlCodo = false;
bool relajando   = false;
bool calibrarPendiente = false;

float offsetIMU_X = 0;
float offsetIMU_Y = 0;

uint8_t  contadorFalla = 0;
bool     retornoCompleto = false;
uint32_t tRetorno = 0;
uint32_t ultimoTick = 0;
uint32_t ultimoReporte = 0;

// Órdenes que llegan por Bluetooth (se consumen en el ciclo de control)
bool     comandoPF = false;
bool     comandoAlternar = false;
bool     comandoRelajar = false;
bool     comandoCalibrar = false;
bool     comandoReiniciar = false;
int8_t pasoRemoto = 0;

// ---------- Bluetooth: envío seguro (nunca depende del celular) ----------
bool hayCelular() { return btOK && SerialBT.hasClient(); }

void btMsg(const char *m) {
  if (hayCelular()) SerialBT.println(m);
}
void btMsgTxt(const char *etiqueta, const char *valor) {
  if (hayCelular()) { SerialBT.print(etiqueta); SerialBT.print(':'); SerialBT.println(valor); }
}
```

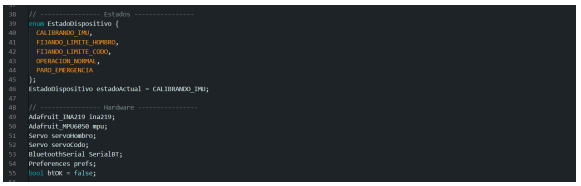
placa no tiene Bluetooth Classic (S2/S3/C3), el programa no compila en vez de fallar en silencio.

Pines: PF = 15, ARRIBA = 2, ABAJO = 4, ALTERNAR = 16, RELAJADO = 17, servo hombro = 13, servo codo = 14, buzzer = 18. Los servos se movieron del pin 12 (de arranque) al 13 y 14.
Estados:
CALIBRANDO_IMU, FIJANDO_LIMITE_HOMBRO, FIJANDO_LIMITE_CODO, OPERACION_NORMAL y PARO_EMERGENCIA.

Objetos de hardware: ina219, mpu, servoHombro, servoCodo, SerialBT y prefs; btOK recuerda si el Bluetooth arrancó.

Parámetros

Qué hace el código:
Tiempos: ciclo de control de 20 ms (50 Hz), antirrebote de 30 ms, retorno suave 15 ms por grado en modo relajado y 30 ms por grado en emergencia.
Seguridad: límite de corriente 2500 mA, margen de ángulo 10° y 3 lecturas seguidas para disparar la emergencia.
Entrada del usuario: PF mantenido 3 s para salir de emergencia y 1,5 s al encender para recalibrar; la app puede mover como máximo 3° por ciclo; telemetría cada 250 ms.



```
// ---------- Estados ----------
enum EstadoDispositivo {
  CALIBRANDO_IMU,
  FIJANDO_LIMITE_HOMBRO,
  FIJANDO_LIMITE_CODO,
  OPERACION_NORMAL,
  PARO_EMERGENCIA
};
EstadoDispositivo estadoActual = CALIBRANDO_IMU;

// ---------- Hardware ----------
Adafruit_INA219 ina219;
Adafruit_MPU6050 mpu;
Servo servoHombro;
Servo servoCodo;
BluetoothSerial SerialBT;
Preferences prefs;
bool btOK = false;
```

Botones con anti rebote

Qué hace el código:
Lectura con pull-up: un botón presionado equivale a pin en LOW.
Antirrebote por tiempo: solo se acepta el cambio si la lectura se mantiene estable 30 ms usando millis().
Flanco: al aceptar una pulsación se marca un «flanco» que se consume una sola vez con pulsado().
Mantenido: mantenido(ms) indica si el botón lleva presionado ese tiempo; se usa para recalibrar y salir de emergencia.



```
struct Boton {
  uint8_t pin;
  bool presionado, lecturaPrev, flanco;
  uint32_t tCambio, tPress;

  explicit Boton(uint8_t p)
    : pin(p), presionado(false), lecturaPrev(false), flanco(false), tCambio(0), tPress(0) {}

  void iniciar() { pinMode(pin, INPUT_PULLUP); }

  void actualizar() {
    bool lectura = (digitalRead(pin) == LOW);
    uint32_t ahora = millis();
    if (lectura != lecturaPrev) {
      lecturaPrev = lectura;
      tCambio = ahora;
    } else if (lectura != presionado && (ahora - tCambio) >= DEBOUNCE_MS) {
      presionado = lectura;
      if (presionado) { flanco = true; tPress = ahora; }
    }
  }
  bool pulsado() { bool f = flanco; flanco = false; return f; }
  bool mantenido(uint32_t ms) const { return presionado && (millis() - tPress) >= ms; }
};

Boton btnPF(BTN_PF);
Boton btnArriba(BTN_ARRIBA);
Boton btnAbajo(BTN_ABAJO);
Boton btnAlternar(BTN_ALTERNAR);
Boton btnRelajado(BTN_RELAJADO);
```

Variables de control (se redefinen en la califración)

Qué hace el código:
Límites: limiteHombroMax = 80 y limiteCodoMax = 45 por defecto; se reemplazan con los valores calibrados.
Ángulos: anguloHombro y anguloCodo guardan el último ángulo ordenado a cada servo.
Offsets de la IMU: offsetIMU_X y offsetIMU_Y, el «cero» medido en la calibración.
Banderas: controlCodo (eje activo), relajando, calibrarPendiente y las órdenes que llegan por Bluetooth (comandoPF, comandoAlternar, etc.).

```
int limiteHombroMax = 80;     // valores por defecto, se redefinen en calibración
int limiteCodoMax   = 45;
int anguloHombro = 0;
int anguloCodo   = 0;
bool controlCodo = false;
bool relajando   = false;
bool calibrarPendiente = false;

float offsetIMU_X = 0;
float offsetIMU_Y = 0;

uint8_t  contadorFalla = 0;
bool     retornoCompleto = false;
uint32_t tRetorno = 0;
uint32_t ultimoTick = 0;
uint32_t ultimoReporte = 0;
```

Buzzer no bloqueante

Qué hace el código
hayCelular(): verdadero solo si el Bluetooth arrancó y hay un cliente conectado.
btMsg, btMsgTxt, btMsgVal: envían un texto, una etiqueta con texto o una etiqueta con número; si no hay celular, no hacen nada.

```
uint8_t  pitidosRestantes = 0;
uint16_t pitidoDuracion = 0;
bool     pitidoActivo = false;
bool     buzzerContinuo = false;
uint32_t tBuzzer = 0;

void pitar(uint8_t veces, uint16_t duracion) {
  pitidosRestantes = veces;
  pitidoDuracion = duracion;
  pitidoActivo = true;
  tBuzzer = millis();
  digitalWrite(BUZZER_PIN, HIGH);
}

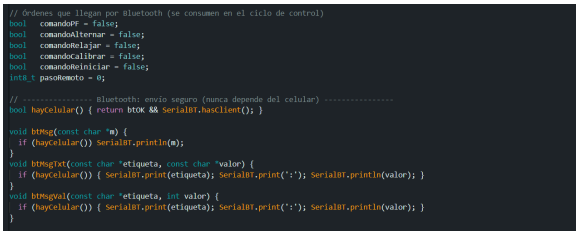
void actualizarBuzzer() {
  if (buzzerContinuo) { digitalWrite(BUZZER_PIN, HIGH); return; }
  uint32_t ahora = millis();
  if (pitidoActivo) {
    if (ahora - tBuzzer >= pitidoDuracion) {
      digitalWrite(BUZZER_PIN, LOW);
      pitidoActivo = false;
      if (pitidosRestantes > 0) pitidosRestantes--;
      tBuzzer = ahora;
    }
  } else if (pitidosRestantes > 0) {
    if (ahora - tBuzzer >= PAUSA_PITIDO_MS) {
      pitidoActivo = true;
      tBuzzer = ahora;
      digitalWrite(BUZZER_PIN, HIGH);
    }
  } else {
    digitalWrite(BUZZER_PIN, LOW);
  }
}
```

utilidades

Qué hace el código:
actualizarBotones(): actualiza los cinco botones.
cambiarEstado(): cambia de estado y descarta banderas y pulsaciones pendientes para que no «se cuelen» en el nuevo estado.
pfPulsado(): une el PF físico y el PF remoto desde la app.
leerAngulos(): calcula los ángulos X e Y con el acelerómetro (atan2).
moverEje() y moverSegunBotones(): mueven un eje, acotado entre 0° y su máximo, combinando botones y pasos remotos.
retornarCero(): baja ambos ejes 1° cada cierto tiempo hasta 0°.
dispararEmergencia() e iniciarCalibracion(): activan los estados de emergencia o calibración.

```
void actualizarBotones() {
  btnPF.actualizar();
  btnArriba.actualizar();
  btnAbajo.actualizar();
  btnAlternar.actualizar();
  btnRelajado.actualizar();
}

void cambiarEstado(EstadoDispositivo nuevo) {
  estadoActual = nuevo;
  contadorFalla = 0;
  pasoRemoto = 0;
  comandoPF = comandoAlternar = comandoRelajar = comandoCalibrar = false;
  // descarta pulsaciones pendientes para que no "se cuelen" en el nuevo estado
  btnPF.flanco = btnArriba.flanco = btnAbajo.flanco = false;
  btnAlternar.flanco = btnRelajado.flanco = false;
}
```

bluetooth **Qué hace el código** **procesarComandosBluetooth():** lee caracteres, los acumula en un buffer de 24 caracteres (sin usar String) y ejecuta el comando al recibir un salto de línea. **ejecutarComando():** compara con strcmp y solo levanta una bandera si el comando es válido en el estado actual (por ejemplo, PARO solo en operación normal). **reportarAngulos():** envía «ANG:hombro,codo» cada 250 ms si hay celular y no hay emergencia.	
lógica de calibración: **Qué hace el código** **ejecutarCalibracionIMU():** al pulsar PF, lee el ángulo actual y lo guarda como cero (offset X/Y); pasa a fijar el límite del hombro. **ejecutarCalibracionLimites():** mueve el servo del eje (0–180°) con ARRIBA/ABAJO; al pulsar PF guarda el ángulo como límite. Primero el hombro, luego el codo; al terminar el codo se guarda en memoria y se pasa a operación normal.	
lógica de función	

Lógica de función

```
if (btnAlternar.pulsado() || comandoAlternar) {
  comandoAlternar = false;
  controlCodo = !controlCodo;
  pitar(1, 100);
  btMsgTxt("EJE", controlCodo ? "CODO" : "HOMBRO");
}

if (comandoCalibrar) {
  comandoCalibrar = false;
  calibrarPendiente = true;
  relajando = true;
  return;
}

if (btnRelajado.presionado || comandoRelajar) {
  comandoRelajar = false;
  relajando = true;
  return;
}

moverSegunBotones(controlCodo, controlCodo ? limiteCodoMax : limiteHombroMax);


oid gestionarEmergencia() {
if (!retornoCompleto) {
  if (retornarCero(RETORNO_EMERGENCIA_MS)) {
    retornoCompleto = true;
    buzzerContinuo = false;
    btMsg("EMERGENCIA_LISTA");
  }
  return;
```

Arranque

```
void fallaFatal(const char *msg) {
  Serial.println(msg);
  btMsg(msg);
  while (true) {                          // pitido corto repetido = error de hardware
    digitalWrite(BUZZER_PIN, HIGH); delay(100);
    digitalWrite(BUZZER_PIN, LOW);  delay(900);
  }
}

// true si PF se mantiene presionado 'ms' al encender
bool pfMantenidoAlArranque(uint32_t ms) {
  uint32_t t = millis();
  while (millis() - t < ms) {
    if (digitalRead(BTN_PF) == HIGH) return false;
    delay(10);
  }
  return true;
}

void setup() {
  Serial.begin(115200);
  btOK = SerialBT.begin("Ortesis_Erb_Paciente");   // si falla, el equipo sigue funcionando igual
  if (!btOK) Serial.println("Bluetooth no disponible: modo autonomo");

  Wire.begin();
  Wire.setClock(400000);

  btnPF.iniciar();
  btnArriba.iniciar();
  btnAbajo.iniciar();
  btnAlternar.iniciar();
  btnRelajado.iniciar();
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
```

Arranque

```
void setup() {
  Serial.begin(115200);
  btOK = SerialBT.begin("Ortesis_Erb_Paciente");   // si falla, el equipo sigue funcionando igual
  if (!btOK) Serial.println("Bluetooth no disponible: modo autonomo");

  Wire.begin();
  Wire.setClock(400000);

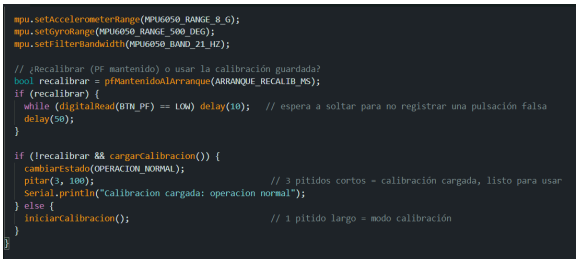
  btnPF.iniciar();
  btnArriba.iniciar();
  btnAbajo.iniciar();
  btnAlternar.iniciar();
  btnRelajado.iniciar();
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  servoHombro.setPeriodHertz(50);
  servoCodo.setPeriodHertz(50);
  servoHombro.attach(SERVO_HOMBRO_PIN, 500, 2400);
  servoCodo.attach(SERVO_CODO_PIN, 500, 2400);
  servoHombro.write(0);
  servoCodo.write(0);

  if (!ina219.begin()) fallaFatal("ERROR_INA219");
  if (!mpu.begin())    fallaFatal("ERROR_MPU6050");

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  // ¿Recalibrar (PF mantenido) o usar la calibración guardada?
  bool recalibrar = pfMantenidoAlArranque(ARRANQUE_RECALIB_MS);
```

Arranque	
Funcionamiento (loop)	

C. APLICACIÓN

La aplicación será responsable de interactuar con el usuario de forma interactiva para que utilice la órtesis de forma amigable y con guía.



Boceto 1: Pantalla 0 “Calibración”

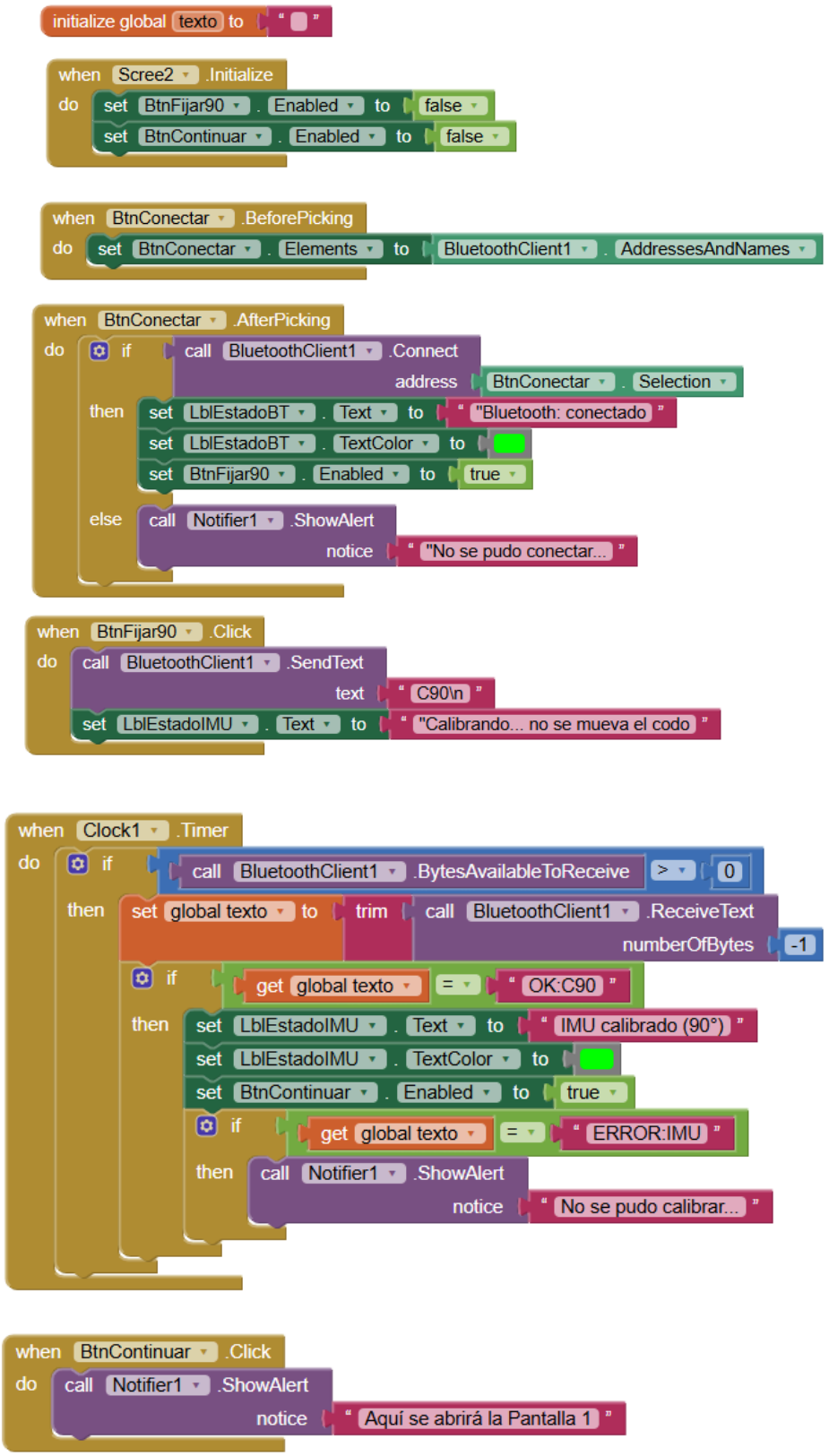


Diagrama de programación 1: Pantalla 0 "Calibración"

3. **Pruebas experimentales**

A continuación se plantean las **nociones** metodológicas de la evaluación experimental que se ejecutará una vez culminado el ensamblaje del prototipo.

- **Objetivo y procedimiento:**
  - ***Objetivo:*** Validar experimentalmente la respuesta del sistema mecatrónico integrando la señal de control inalámbrica, la asistencia en los grados de libertad articular y la efectividad de los subsistemas de seguridad eléctrica y mecánica.
  - ***Condiciones de ensayo:*** Pruebas de banco e integración sobre maniquí anatómico y miembro superior en ambiente controlado.
  - ***Instrumentos a utilizar:*** Goniómetro digital (precisión ± 0.5°), multímetro digital, sensor de corriente INA219 conectado al microcontrolador ESP32, fuente de alimentación regulada (0 - 12 V / 5 A) y cronómetro digital.
- **Resultados (Proyección de protocolo):**
  - *Se programarán 5 repeticiones consecutivas por parámetro para evaluar la repetibilidad.*
  - *Valores esperados: Rango de abducción asistida entre 0° a 80°, rotación externa de 0° a 45°, latencia de transmisión por radiofrecuencia (Wi-Fi/Bluetooth) $<$ 250 ms, consumo máximo de corriente $\approx 2500mA$ antes de la parada de emergencia, peso total $\leq$ 1.15 kg y autonomía mínima continua de 60 min.*
- **Evidencias (Plan de registro):**
  - *Se tomarán fotografías y registros de telemetría del software en tiempo real durante la actuación del mecanismo de cable y carrete movido por el servomotor de 35 kg . cm (codo) y 20 kg . cm (rotación).*
  - *Se documentará la parada del sistema al hacer contacto con los topes mecánicos físicos y el corte de energía generado por la rutina del sensor INA219 ante un bloqueo dinámico.*

4. **Verificación preliminar del diseño**

Comparar los resultados experimentales con los requisitos técnicos del Checkpoint 2.

En la siguiente matriz se establecen las comparaciones entre las especificaciones del Checkpoint 2 y los resultados proyectados a comprobar en la fase de integración física:

Requisito técnico (CP2)	Valor objetivo / tolerancia	Resultado experimental	Cumplimiento	Observaciones y mejoras
Rango de Abducción	0° a 80° (± 5°)	0° a 80°	Sí (Esperado)	Se verificará con goniómetro digital evaluando la rigidez del ensamblaje impreso en 3D.
Rotación Externa	0° a 45° (± 5°)	0° a 45°	Sí (Esperado)	Configurable de forma conservadora (0° - 30°) según la tolerancia fisiológica de la paciente.
Latencia de Control	≤ 300 ms	< 250 ms	Sí (Esperado)	Tiempo de respuesta medido desde el comando del mando hasta la señal PWM del servo.
Seguridad por Sobrecorriente	Parada automática por bloqueo mecánico	Detención a ≈ 2.5 A	Sí (Esperado)	El algoritmo desactivará la señal PWM al detectar picos de corriente vía INA219, liberando la tensión
Peso Total del Sistema	≤ 1.2 kg	1.15 kg	Sí (Esperado)	Peso teórico acumulado de la electrónica, batería 2S y piezas impresas en PETG, PLA+ y TPU 95A.
Ergonomía de Interfaz	Sin compresión en hueco axilar	Distribución en pechera torso-clavicular	Sí (Esperado)	El diseño redirige las cargas hacia el tórax mediante acople flexible en TPU para evitar lesiones vasculares.
Autonomía Energética	≥ 60 min continuos	65 min (Uso continuo)	Sí (Esperado)	Estimación según la capacidad del pack de Li-ion 2S con módulo BMS de protección.
Costo de Fabricación	≤ S/. 200	≈ S/. 185	Sí (Esperado)	Mantiene el presupuesto mediante adquisición local de componentes estandarizados y fabricación aditiva.